Tools — Sub-Agents

Spawn focused sub-agents that run synchronously or in the background. Prefer sub-agents over doing work inline: delegate exploration, builds, and planning.

Tool	Description
`new_agent`	Spawn a sub-agent (blocking or background).
`cancel_agent`	Cancel a running background sub-agent.
`list_agents`	List sub-agent tasks for the session.
`wait_agents`	Block until background tasks complete.
`agent_complete`	Terminal signal: the autonomous task is done.

System instruction: the `## Sub-Agent Spawning` block a primary agent receives teaches the whole contract, in the same words as the tool descriptions below:

- **Three execution contracts** - `foreground` (`background` absent or `false`), `background` (`background: true, detached: false`) and `detached` (`background: true, detached: true`), with the selecting shape named for each. `detached: true` without `background: true` names no contract and is refused.
- **The limit and its occupancy** - the configured number, counting `running`, `suspended` and `terminating` background tasks and never a detached or foreground one, plus the cancel-to-free rule: *to free a slot, CANCEL - do not suspend*.
- **The await contract** - `wait_agents` blocks until tasks settle, every settled task reports its canonical registry status (`completed` / `failed` / `cancelled` / `timed_out`, so a cancel is never reported as a failure), `suspended` is not settled while `timed_out` is, and a detached task can never be awaited.
- **The durable report path** - `log/subagents/<task-id>.md`, surfaced as `output_file`, with the in-memory-only case named when no durable copy exists.

A sub-agent receives the completion protocol instead: it cannot spawn, so the block is scoped to primary agents.

Available agents: `explore` (fast, read-only codebase understanding), `build` (compile/test/fix), `plan` (implementation plans, read-only), `general` (full-capability fallback).

Strict parameters

Every tool in this family declares a closed parameter set (`additionalProperties: false`) and actually enforces it: a call carrying a property the tool does not declare fails and names the offending key, instead of silently running with the default. `new_agent agent="explore" task="..." backgound=true` is refused as `Unknown parameter(s): `backgound`` rather than silently running a *foreground* task.

The parameter check runs before the environment check, so a bad call shape is reported as a bad call shape even when no agent manager is wired in.

`bg` additionally refuses a declared parameter that the selected action does not read: `{"action": "status", "task_id": "...", "lines": 10}` names `lines` as not accepted by `status` rather than ignoring it.

Refusals are errors

A precondition failure is a **tool error**, never a successful result whose content is prose, so a model cannot mistake a refusal for an answer.

Inside an active team, `new_agent` is refused with `team_context_active`: a lead is pointed at `team_spawn`, `team_task_create`, `team_assign_task` and `team_message`; a teammate is pointed at `team_read_messages`, `team_task_claim`, `team_task_complete` and `team_idle`. No sub-agent task starts.

Permission categories

The four tools read and write one object - the session's task registry - so they share one taxonomy, and no tool in the family is uncategorised:

Tool	Category	Meaning
`new_agent`	`agent:spawn`	Creates a task.
`cancel_agent`	`agent:control`	Controls a task that already exists.
`wait_agents`	`agent:control`	The same control action, blocking.
`list_agents`	`agent:read`	Reads the registry; changes nothing.

Three separate categories mean a `permission` rule can grant spawning while denying cancellation, or deny observation without denying the spawn. All four remain hardwired auto-approved in an interactive session, so the classification tightened without adding a prompt.

Permissions inside a sub-agent run

A sub-agent run has no attached user, so it could never answer an interactive prompt - the call would wait out the watchdog and the run would be lost. A call by an unattended run whose tool can **write** therefore skips the prompt, regardless of the global `/yolo` flag (SPEC 4.7.1; see permissions).

- A tool is write-capable unless its `permission_category()` is read-only (`file:read`, `codeindex`, `web`, `network`, `model`, …). An uncategorised tool counts as write-capable.
- A read-only run - `explore`, a plan reviewer, a loop restricted to reads - gets no automatic scope at all and keeps the ordinary prompt policy.
- Only the *prompt* is skipped. The loop tool-set restriction, the path guards, the bash `is_safe_command` stage, the tool-repeat guard, every pre-tool hook and an explicit policy `Deny` still apply, and a forced destructive-action checkpoint still prompts.
- Interactive tools (`AskUserQuestion`) are filtered from a sub-agent's tool surface and denied at dispatch, independent of the scope above.

---

new_agent

Spawn a sub-agent to perform a focused task. **Both `agent` and `task` are required.** The agent is stateless — batch all related questions into one call with a comprehensive prompt.

Arguments

Argument	Type	Required	Description	Typical value
`agent`	string	yes	Agent name (`explore`, `build`, `plan`, `general`, or custom)	`"explore"`
`task`	string	yes	Specific prompt/instructions, including all needed context	`"Find all callers of X in src/"`
`background`	boolean	no	Run concurrently without blocking (default false). Use `true` whenever spawning more than one in the same response	`true`
`detached`	boolean	no	Only with `background: true`. Fire-and-forget: excluded from `list_agents`, not awaitable via `wait_agents`, completion NOT injected back into the session	`true`
`model`	string	no	Model override, `provider/model` (preferred) or `provider:model`	`"anthropic/claude-sonnet-4-20250514"`
`timeout_secs`	integer	no	Per-call wall-clock bound (>= 1) for the spawned task. It **tightens** the configured stall timeout and never extends it; exceeding it ends the task as `timed_out`	`600`

Every argument above is named in the tool's own description, and `test_tool_descriptions` fails the build if a schema property and its description drift apart in either direction - a declared parameter with no sentence, or a sentence advertising a parameter the schema would refuse.

Concurrency: at most 32 background tasks per session; use `wait_agents` to free slots before spawning more. A **suspended** background task still counts against the limit - suspending does not free a slot, only cancelling does (see *Occupancy* below).

A **detached** task (`background: true, detached: true`) runs and shows in the Agents panel but is in-

visible to the delegation surface: `list_agents` omits it, `wait_agents` (with or without `task_ids`) never returns it, and its completion is reaped without a chat injection while still appearing in `tasks_snapshot`. Because nothing reads its reply body, a detached prompt whose deliverable is a report MUST name a file to write; the run's full output is additionally persisted to `log/subagents/<task-id>.md` on completion. The TUI `/spawn` command is the user-facing equivalent.

Example:

```
new_agent agent="explore" task="Summarise architecture in crates/ragent-agent/src/agent/" backgro
wait_agents
```

---

cancel_agent

Cancel a running background task spawned with `new_agent background=true`. Not for team tasks.

A successful call means the cancel was **requested**, not that the task is finished. The result reports `cancel_requested: true` plus the task's **current** status (`running`, `suspended`, or `terminating`), and the terminal `cancelled` status arrives later - poll `list_agents` or await it with `wait_agents`. The tool never reports `cancelled` for a request the registry has not yet acted on.

If the registry rejects the request (the task's cancel flag is already gone), the result reports `reason: cancel_failed`, `cancel_requested: false`, and the status the task is actually in, so "already completed" is not indistinguishable from a delivery failure. `status` is `unknown` only when the entry could not be read back at all.

Arguments

Argument	Type	Required	Description
`task_id`	string	yes	Task identifier returned by `new_agent`

The TUI's `/cancel <prefix>` slash command calls the same `AgentManager::cancel_agent` method, so a cancellation requested from the input line and one requested by the tool have identical semantics. The command adds prefix resolution: an ambiguous prefix is refused with every match listed, and an unmatched prefix lists the live task ids. See `/cancel`.

---

list_agents

List sub-agent tasks for the current session with status and result summary; completed tasks include an `output_file` path with the FULL untruncated report under `log/subagents/<task-id>.md`.

A task that in-flight `wait_agents` callers are holding reports how many (`waiter_count`) in the listing row, in the `task_id` detail view (`Active Waiters: N`) and in the metadata. The count is reported **only when it is greater than zero**, so a task nobody awaits shows no waiter count at all rather than a `0`. It follows the registry's own tally, so it disappears the moment the wait returns, and a suspended task still shows its queued waiters.

Arguments

Argument	Type	Required	Description	Typical value
`status`	enum	no	Filter by `running` / `suspended` / `completed` / `failed` / `cancelled` / `timed_out` (`terminating` is reachable but unlisted; read it with `task_id`)	—
`task_id`	string	no	Get details for one task	—

---

wait_agents

Block until one or more background tasks complete (preferred over polling `list_agents`). For team members use `team_wait` instead.

Arguments

Argument	Type	Required	Description	Typical value
`task_ids`	array	no	Specific tasks to await (default: all running)	`["task-abc"]`
`timeout_secs`	integer	no	Max seconds to wait before returning partial results	`300`

A **suspended** task is still awaited: only a terminal status settles a wait, so a suspended task keeps the waiter blocked until it is resumed and finishes, or is cancelled.

With `task_ids` omitted the wait is seeded from the registry's **running** background tasks, so it reports `No running background tasks to wait for` only when none are actually working. (Version 1.0.153 regressed this into a settled-only scan, which answered with that message while agents were still running; 1.0.165 restores the running-task seed.)

One definition of "finished"

`TaskStatus::is_terminal()` is the single classification the whole sub-agent lifecycle shares:

Status	Terminal?	Meaning
`completed`	yes	The run ended and produced a result
`failed`	yes	The run ended in error

Status	Terminal?	Meaning
`cancelled`	yes	The cancel request was observed and the run stopped
`timed_out`	yes	The per-call wall-clock bound was hit
`running`	no	Working right now
`suspended`	no	Parked, resumable, still holds its slot
`terminating`	no	A kill is in flight

Reaping (`drain_completed`), result injection, kill refusal, `wait_agents`, `team_wait` and the Agents panel all consult it, so a new status cannot be classified as finished on one path and live on another. Before 1.0.165 the drain treated anything that was not `running` as settled: it injected a *suspended* or *terminating* task as an empty completion and then removed it, so a later resume failed with "not found" and the real result was lost.

The same predicate closes three adjacent holes:

- `kill_task` refuses a `timed_out` task instead of re-killing an ended run;
- `suspend_task` / `resume_task` check and set the status under one guard (and store the park flag with it), so a task that finished in between is not overwritten back to `suspended`;
- `cancel_all` also cancels suspended tasks, and a finished foreground (`spawn_sync`) entry is reaped by the next drain instead of living for the process lifetime. `spawn_sync` returns the snapshot taken under the completion guard, so a concurrent drain cannot reap the entry it just reported and turn a successful run into "vanished after completion".

`Event::SubagentComplete` carries the terminal `status` and `output_file` alongside `success`, so the Agents panel renders the outcome and the durable report path without a second registry lookup that could race the drain.

Occupancy: a suspended task still holds its slot

The background limit counts a task from the moment it is spawned until it reaches a terminal status (`completed`, `failed`, `cancelled`, `timed_out`). A **`suspended`** background task is counted exactly like a `running` one, because it keeps its `background_timeout_secs` wall-clock budget and can resume at any time: letting it free a slot would let the resume push occupancy silently over the limit.

So suspending a task frees **no** headroom. `list_agents` reports the limit, the `occupancy` and the `headroom` on every call, and a `new_agent` refused for the limit lists the occupying task ids. To free a slot:

1. `wait_agents` - wait for a running task to finish.
2. `cancel_agent` - cancel one, which is the supported way to free a slot early.

Foreground and `detached` tasks never occupy a slot.

---

agent_complete

Terminal signal — the current autonomous task is fully done. Ends the session loop and returns control to the user. Only call when all requested outputs have been produced.

Arguments

Argument	Type	Required	Description
`summary`	string	yes	Concise summary of what was accomplished

Do NOT pass `task_id`, `team_name`, `result`, or `output`; to mark a *team* task done, use `team_task_complete` inside a team.